



SCHEDULING & NODE MANAGEMENT

Book 04 of the series — inside the Scheduler's decision: filter, score, bind. The full taints & tolerations system, node/pod affinity, topology spread, priority & preemption, and the maintenance discipline: cordon, drain, static Pods.



Scheduler

filter → score → bind



Taints

all 3 effects



Tolerations

the keys



Affinity

node & pod



Spread

zones & skew



Priority

+ preemption



Drain

maintenance



Static Pods

kubelet-born

PART I · THE SCHEDULER'S MIND

- ☐ **CH 01** How Scheduling Works p.3
- ☐ **CH 02** The Scheduling Toolbox p.4

PART II · TAINTS & TOLERATIONS — FULL SYSTEM

- ☐ **CH 03** Taints — Effects & Use Cases p.5
- ☐ **CH 04** Tolerations — Operators & tolerationSeconds p.6
- ☐ **CH 05** Well-Known Taints — The Cluster's Own p.7

PART III · AFFINITY & SPREAD

- ☐ **CH 06** nodeSelector & Node Affinity p.8
- ☐ **CH 07** Pod Affinity & Anti-Affinity p.9
- ☐ **CH 08** topologySpreadConstraints p.10

PART IV · PRIORITY & EVICTION

- ☐ **CH 09** Priority Classes & Preemption p.11

PART V · NODE OPERATIONS

- ☐ **CH 10** Cordon, Drain & Uncordon p.12
- ☐ **CH 11** Static Pods p.13

PART VI · LABS

- ☐ **LAB 1** Node Affinity — Required vs Preferred p.14
- ☐ **LAB 2** Anti-Affinity — Prove the Separation p.15
- ☐ **LAB 3** Priority & Preemption Live p.16
- ☐ **LAB 4** The Maintenance Cycle p.17

PART VII · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.18
- ☐ **KEY** Answer Key + Scoring p.19
- ☐ **GLOSS** Glossary p.20
- ☐ **CHEAT** One-Page Cheat Sheet p.21
- ☐ **FINALE** Where to Next → Book 05 p.22

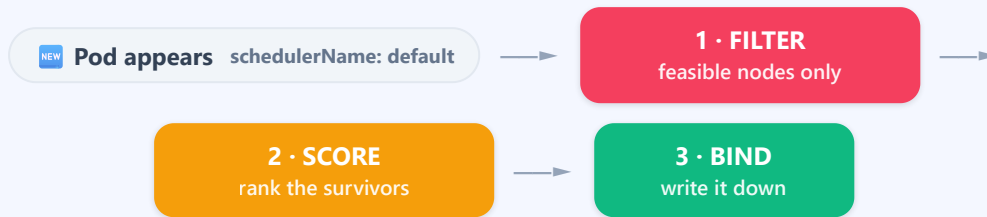


Assumes Books 01–03. Book 01 · CH 07 + Lab 5 gave you taints' first taste; CH 23 showed Pending = "the Scheduler said no". This book opens the black box: *every reason the Scheduler says yes or no, and every knob you own.*



The Scheduler is a control loop (Book 01 · CH 02) with one job: for each **unscheduled** Pod, pick the best feasible Node and **bind** the Pod to it. Every decision is a three-stage funnel:

THE FUNNEL — EVERY POD, EVERY TIME



filter kills nodes that *can't* take the Pod · score ranks nodes that *can* · the highest score wins

1 · FILTER — HARD CONSTRAINTS

Pod's **requests** fit the node's allocatable (Book 02 · CH 14)

Taints tolerated? (Part II of this book)

nodeSelector / required affinity match?

Ports, volumes, hostNetwork conflicts

Zero nodes survive → Pod stays Pending. This is the #1 cause.

2 · SCORE — SOFT PREFERENCES

Preferred affinity/anti-affinity weights

Spreading vs packing balance (CH 08)

Image locality (image already pulled?), pod spread across zones...

Every plugin votes 0–100; sum decides. **Ties** → **random**.

```
# the binding is a real API object – you can see the decision:
$ kubectl get pod web -o jsonpath='{.spec.nodeName}'
minikube
```



Scheduling is one-time, not a lease: once bound, the Scheduler never reconsiders. If the node degrades, the **kubelet** and node-lifecycle controller react (taints → evict) — the Scheduler only picks *new* placements. Re-scheduling happens only if the Pod *dies and is recreated*.



One sentence: *Filter decides IF a node can; Score decides WHICH node should; Bind makes it official* — and every tool in this book plugs into exactly one of those stages.

▶▶ The full knob inventory — [CH 02: The Scheduling Toolbox](#)

↑ back to top



Five mechanisms decide placement. Knowing *which stage* each one acts in tells you which to reach for:

Tool	Stage	Hard or soft?	One-line job
Resources (requests)	Filter + Score	hard (capacity)	"this Pod needs 500Mi / 250m somewhere"
Taints + Tolerations	Filter	hard (node repels)	"keep ordinary Pods OFF this node"
Node affinity / nodeSelector	Filter / Score	both (required / preferred)	"this Pod WANTS certain nodes"
Pod (anti-)affinity	Filter / Score	both	"co-locate / separate from other Pods"
topologySpread	Score (+ filter)	soft or hard	"spread evenly across zones/nodes"



taints node pushes away



affinity pod pulls toward



spread distribution discipline



Direction matters — the classic confusion: taints are **node-side repulsion** (the node refuses visitors without keys). Affinity is **pod-side attraction** (the Pod states its preferences). They never do each other's job — taints can't attract, affinity can't evict.

**Dedicate nodes**

GPU / memory-huge
→ taints + tolerations

**Pin to hardware**

SSD, region, zone
→ node affinity

**Resilience layout**

HA across zones
→ spread + anti-affinity



Decision shortcut: keeping things **OFF** nodes → taints. Getting things **ON** specific nodes → node affinity. Relationships **between Pods** → pod affinity/anti-affinity. **Evenness** → spread. The rest of the book is these four, in depth.

Part II opens with the repulsion side — [CH 03: Taints — Effects & Use Cases](#)

↑ back to top



A taint is a **node-side repulsion field**: `key=value:effect`. Book 01's Lab 5 proved the mechanics — this page is the full strategic picture.

```
$ kubectl taint node node1 gpu=nvidia:NoSchedule
node/node1 tainted
$ kubectl taint node node1 gpu=nvidia:NoSchedule- # trailing dash = remove
```

Effect	Running Pods	New Pods	Use case
NoSchedule	stay	blocked unless tolerated	dedicate nodes (GPU, SSD, prod-only)
PreferNoSchedule	stay	avoided if alternatives exist	soft segregation — "try not to"
NoExecute	EVICTED now	blocked unless tolerated	remove workloads when a node degrades

The three strategic uses

1 · Dedication

`dedicated=ml:NoSchedule` — only ML team's Pods (which tolerate) may land. A reservation *without* reserving capacity.

2 · Special hardware

`gpu=nvidia:NoSchedule` — prevents accidental placement of Pods that can't use the GPU... and of Pods that would *waste* it.

3 · Node condition → eviction

The control plane itself taints sick nodes with **NoExecute** — workloads flee a dying node automatically (CH 05).



Two traps: (1) taints don't **attract** — a tolerated Pod *may* land elsewhere; to pin it, combine taints with **node affinity** (CH 06). (2) **NoExecute is a weapon** — adding one manually evicts production Pods mid-flight. Book 01 · Lab 5's trick question (taint *after* Pod exists) is this effect in action.



One sentence: taints come in three strengths — *block new* (**NoSchedule**), *gently avoid* (**PreferNoSchedule**), or *evict existing* (**NoExecute**) — and their job description is "keep the wrong workloads off this node."

The key side of the lock — [CH 04: Tolerations](#)

↑ back to top

Tolerations — Operators & tolerationSeconds

A toleration is a **Pod-side permission**: "I may land on (or survive on) nodes carrying this taint." Match rule: **key** + **operator** (+ **value**) + **effect**.

```
spec:
  tolerations:
  - key: gpu
    operator: Equal          # default – value must match exactly
    value: nvidia
    effect: NoSchedule
  - key: "dedicated"
    operator: Exists        # ANY value, as long as key exists
    effect: NoSchedule
  - key: ""
    operator: Exists
    effect: ""              # tolerate EVERYTHING (use with fear)
```

Operator	Matches	Value field
Equal default	key exists and value is identical	required
Exists	key exists, any value (wildcard)	must be empty

🕒 tolerationSeconds — the grace timer (NoExecute only)

```
tolerations:
- key: node.kubernetes.io/not-ready
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 300 # if the taint lasts > 5 min → THEN evict me
```



The resilience dial: a transient blip (< 5 min) → your Pod rides it out. A real outage → eviction after the grace period. This is *exactly* how the default tolerations on Deployments behave — it's why brief node hiccups don't mass-evict your fleet.

🚫 No `tolerationSeconds` + NoExecute tolerated = **survive forever** on the tainted node

⚙️ Effect matching is literal: a toleration for `NoSchedule` does nothing against `NoExecute` on the same key



One sentence: tolerations are the **only** way past a taint — `Equal` for exact locks, `Exists` for skeleton keys, and `tolerationSeconds` turns a hard eviction into a grace period.

🔗 The taints you didn't create — [CH 05: Well-Known Taints](#)

↑ back to top



The node-lifecycle controller and kubelet **taint nodes automatically** as conditions change. These aren't your taints — they're the cluster's immune response, and recognizing them is a diagnostic superpower.

Taint	Effect	Meaning
<code>node.kubernetes.io/not-ready</code>	NoSchedule/NoExecute	node's Ready condition = False (kubelet heartbeat lost)
<code>node.kubernetes.io/unreachable</code>	NoExecute/NoSchedule	node unreachable (network partition) — later becomes not-ready
<code>node.kubernetes.io/disk-pressure</code>	NoSchedule	disk running out — no NEW pods (existing keep running)
<code>node.kubernetes.io/memory-pressure</code>	NoSchedule	memory scarcity on the node
<code>node.kubernetes.io/network-unavailable</code>	NoSchedule	networking not configured yet (some clouds)
<code>node.kubernetes.io/unschedulable</code>	NoSchedule	added by <code>kubectl cordon</code> — CH 10
<code>node-role.kubernetes.io/control-plane</code>	NoSchedule	the brain-protection taint (Book 01 · CH 07)

Node degrades → **NotReady** / **Unreachable** NoExecute taint applied

```

|
├─ pods without toleration → evicted (grace per tolerationSeconds)
├─ deployments recreate them on healthy nodes ← Book 01 · CH 06 loop
└─ node recovers → taint removed → schedulable again

```



Debug move: mysterious Pending + "node(s) had untolerated taint" in events (Book 01 · CH 23)? Run `kubectl describe node N | grep Taints` — the taint list is the diagnosis. `disk-pressure` appearing on nodes is also your cue for storage cleanup (Book 05's territory).



One sentence: the cluster self-medicates with taints — *sick nodes get NoExecute'd, workloads evacuate, healthy nodes absorb the load* — and self-healing "magic" from Book 01 is really this mechanism plus reconciliation.

▶ Part III: now the Pod speaks — [CH 06: Node Affinity](#)

↑ back to top



Pod-side attraction to specific nodes. `nodeSelector` is the simple ancestor; **node affinity** adds operators, weights, and soft preferences.

```
spec:
  nodeSelector:           # the simple form: hard, exact-match only
    disk: ssd
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution: # HARD (filter)
        nodeSelectorTerms:
          - matchExpressions:
              - { key: disk, operator: In, values: [ssd, nvme] }
      preferredDuringSchedulingIgnoredDuringExecution: # SOFT (score)
        - weight: 80          # 1-100 — a vote, not a veto
          preference:
            matchExpressions:
              - { key: zone, operator: In, values: [east] }
```

Operators	Meaning
In · NotIn	value is / is not in the list
Exists · DoesNotExist	key present / absent (any value)
Gt · Lt	numeric greater/less than (node labels as numbers)



Decode the long names — they're the whole contract: `required...` = filter (no match → Pending). `preferred...` = score (hint). **IgnoredDuringExecution** = if the node's labels *change after* your Pod is running, the Pod **stays** — affinity is checked at scheduling time only. (The mirror concept `RequiredDuringExecution` exists for pod affinity — live re-evaluation.)

label node: `kubectl label node n1 disk=ssd`

→

Pod with `disk In [ssd]`

→

lands only there



One sentence: node affinity = nodeSelector with superpowers — **operators for matching, weights for preferences, and "IgnoredDuringExecution" meaning the lock is checked once, at scheduling time.** Lab 1 proves the hard/soft split live.

▶ Affinity between Pods — [CH 07: Pod Affinity & Anti-Affinity](#)

↑ back to top



Node affinity asks "which *nodes*?" — pod affinity asks "near which *other Pods*?", measured through a **topology domain** (same node? same zone? same region?).

```
spec:
  affinity:
    podAffinity:
      # CO-LOCATE — "keep me near my cache"
      requiredDuringSchedulingIgnoredDuringExecution:
        - topologyKey: kubernetes.io/hostname # same NODE
          labelSelector:
            matchLabels: { app: cache }
    podAntiAffinity:
      # SEPARATE — "never put replicas together"
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            topologyKey: topology.kubernetes.io/zone # different ZONES
            labelSelector:
              matchLabels: { app: web }
```

topologyKey (common)	Domain = "together means..."
kubernetes.io/hostname	same physical node
topology.kubernetes.io/zone	same availability zone
topology.kubernetes.io/region	same region

✓ AFFINITY (CO-LOCATE)

web + its cache on one node → localhost-fast calls, one bill. **Latency-sensitive pairs** that talk constantly.

✓ ANTI-AFFINITY (SEPARATE)

3 replicas of web on 3 zones → one zone dies, 2/3 survive. **Never stack the blast radius.**



required anti-affinity is a placement cliff: more replicas than topology domains → the extras sit **Pending forever** (3 replicas, 1 node = 1 runs, 2 Pending — Lab 2 proves it). Production default: **preferred + weight 100**, or better — **topologySpread** (CH 08).



One sentence: pod (anti-)affinity = relationship rules between Pods, expressed as "same/different *topology domain*" — powerful for latency and HA, but **required** versions trade flexibility for guarantees.

▶ The modern alternative — [CH 08: topologySpreadConstraints](#)

↑ back to top



Anti-affinity answers "are any Pods here?" — spread answers a better question: **"is the distribution even?"** One constraint replaces a pile of weights.

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1                # max allowed imbalance
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule # or: ScheduleAnyway (soft)
      labelSelector:
        matchLabels: { app: web }
# - maxSkew = (most pods in one domain) - (fewest)
# - domains with ZERO pods count too (they pull new pods!)
```

MAXSKEW: 1 IN ACTION — 6 REPLICAS, 3 ZONES

zone-a
2 🐛🐛

zone-b
2 🐛🐛

zone-c
2 🐛🐛

2-2-2 → skew 0 ✓ · 3-2-1 → skew 2 ✗ next Pod forced to zone-c · 3-3-0 → skew 3, but 0-domain "pulls" until fed

whenUnsatisfiable

Behavior when the rule can't hold

DoNotSchedule

hard — Pod waits (Pending) until it can comply

ScheduleAnyway

soft — place anyway, but the score penalty steers future Pods



Spread vs anti-affinity — when which? Anti-affinity: *boolean* togetherness, per-Pod-pair relationships, cheap on small sets. Spread: *numeric* evenness across N domains, scales to hundreds of replicas, and — the killer feature — counts **empty domains**, so a fresh zone always receives traffic first.



One sentence: topologySpreadConstraints = "keep skew ≤ maxSkew across this topology" — the modern, scalable answer to "spread my replicas," with a hard mode (`DoNotSchedule`) and a soft mode (`ScheduleAnyway`).

🔊 When scarcity gets violent — [CH 09: Priority & Preemption](#)

↑ back to top



When nothing fits anywhere, a high-priority Pod may **evict lower-priority Pods** to make room. Priority is the cluster's pecking order under scarcity.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: high-priority }
value: 1000000          # higher = more important
globalDefault: false    # one class MAY be the default
preemptionPolicy: PreemptLowerPriority # default; or Never
```

```
spec:
  priorityClassName: high-priority # attach to the Pod
```

Built-in class	Value	Reserved for
system-node-critical	2,000,001,000	kubelet-essential: CNI, kube-proxy
system-cluster-critical	2,000,000,000	CoreDNS, metrics — cluster functions
<i>your Pods (no class)</i>	0	everything below the system floor

How preemption actually fires

```
High-prio Pod → Pending (no node fits – filter fails on resources)
  | scheduler tries placement
  ▼
Preemption attempt: find node where evicting lower-prio victims
  would free enough for the whole Pod (incl. its requests!)
  ▼
victims deleted gracefully (their owners recreate them – elsewhere, later)
  ▼
high-prio Pod scheduled into the gap ✓
```



Two honesty notes: (1) preemption is **resource-arithmetic, not revenge** — the scheduler only kills the *minimum set* of lowest-priority victims, and only when the high-prio Pod genuinely can't fit. (2) Your Pods' priority also decides **kubelet eviction order** under node pressure — layered on top of QoS from Book 02 · CH 15.



One sentence: PriorityClass = a number that means "in a shortage, I eat first" — driving both **preemption at scheduling time** and **eviction order at runtime**. Lab 3 watches a real preemption.

▶ Part V: now *you* touch the nodes — [CH 10: Cordon, Drain & Uncordon](#)

↑ back to top



Node maintenance (kernel upgrades, hardware swaps) without downtime = a three-verb dance. **Cordon** stops new arrivals; **drain** evacuates; **uncordon** reopens.

```
$ kubectl cordon node1          # 1 · add unschedulable taint (CH 05)
node/node1 cordoned
$ kubectl drain node1 --ignore-daemonsets --delete-emptydir-data
node/node1 cordoned          # drain cordons for you too
evicting pod shop/web-7d9f    # graceful eviction – PDB-aware!
pod/web-7d9f evicted
$ # ...do the maintenance...
$ kubectl uncordon node1      # 3 · reopen
node/node1 uncordoned
```

Flag	Why it exists
<code>--ignore-daemonsets</code>	DaemonSet Pods are node-bound by design (log agents, CNI) — drain skips them; without the flag drain aborts
<code>--delete-emptydir-data</code>	Pods with <code>emptyDir</code> lose data on eviction — drain refuses by default; the flag says "ephemeral, go ahead"
<code>--force</code>	evict Pods not managed by a controller (standalone Pods die with no replacement — you were warned, Book 01 · CH 15)
<code>--grace-period / --timeout</code>	tune eviction grace; PDBs can block indefinitely → drain waits/times out



Drain respects PodDisruptionBudgets (PDBs) — "keep ≥ 2 web replicas alive" pauses drain until it can evict safely. Production rule: *define PDBs before your first real drain*, or the "safety" is zero. (Also: cordon ≠ drain — cordon alone just stops *new* placements.)



1 cordon — no NEW Pods land (existing keep running)



2 drain — evict everything the controllers can recreate, PDB-respecting



3 maintain — patch, reboot, swap hardware



4 uncordon — taint removed; Scheduler refills the node



One sentence: maintenance = *cordon (close the door) → drain (empty the room, politely) → uncordon (reopen)* — and the two flags you'll type forever are `--ignore-daemonsets` and `--delete-emptydir-data`. Lab 4 runs the full cycle.

[↑ back to top](#)



Static Pods are created and managed **directly by the kubelet**, from manifest files on the node's disk — no Deployment, no Scheduler, no controller. The Scheduler's total bypass.

```
kubelet flags: --pod-manifest-path=/etc/kubernetes/manifests
| watches the dir (default on kubeadm)
```



```
file appears → kubelet runs the Pod      no API call needed
file edited  → Pod updated                 file IS the spec
file deleted → Pod dies
kubectl delete → Pod briefly dies → kubelet recreates it (undeletable!)
```

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	
etcd-minikube	1/1	Running	# ← static!
kube-apiserver-minikube	1/1	Running	# ← static!
kube-controller-manager-minikube	1/1	Running	# ← static!
kube-scheduler-minikube	1/1	Running	# ← static!
coredns-7db6d8ff4b-abcde	1/1	Running	# ← Deployment



The bootstrap paradox, solved: the Control Plane runs as static Pods because *something must run the API Server before the API Server exists*. The kubelet + a folder of YAML boot the entire brain — chicken, meet egg. (The `-minikube` / `-nodename` suffix: kubelet also creates a read-only **mirror Pod** in the API so you can `kubectl get` them.)

Your own static Pod: drop a valid Pod manifest into the manifest dir → running within seconds, Scheduler never involved

They ignore taints, affinity, and drain-eviction logic (the file rules) — that's why `drain --ignore-daemonsets` and static/mirror pods go together



One sentence: static Pods are **kubelet-and-disk only** — the mechanism that boots the control plane itself, undeletable through `kubectl` because the source of truth is a file on the node.

Prove all of it with your hands — [Part VI: Labs](#)

[↑ back to top](#)



STEP 1 · Label the node + hard affinity

```
$ kubectl label node minikube disk=ssd
$ kubectl create deploy aff --image=nginx --replicas=2
$ kubectl patch deploy aff --type=merge -p '{"spec":{"template":{"spec":{"affinity":{"nodeAffinity":{"requiredDuringSchedulingIgnoredDuringExecution":{"nodeSelectorTerms":[{"matchExpressions":[{"key":"disk","operator":"In","values":["ssd"]}]}]}}}}}}'
$ kubectl get pods -o wide # all on minikube ✓ (label matches)
```

STEP 2 · Break the label — watch required bite

```
$ kubectl label node minikube disk-
$ kubectl scale deploy aff --replicas=3
$ kubectl get pods
aff-xxx 1/1 Running # old pods: IgnoredDuringExecution — STAY
aff-yyy 0/1 Pending # new pod: no node matches → filter fails
$ kubectl describe pod -l app=aff | grep -A2 Events | tail -2
0/1 nodes are available: 1 node(s) didn't match ... affinity
```

STEP 3 · Switch to preferred — same break, different outcome

```
$ kubectl delete deploy aff && kubectl create deploy soft --image=nginx --replicas=2
$ kubectl patch deploy soft --type=merge -p '{"spec":{"template":{"spec":{"affinity":{"nodeAffinity":{"preferredDuringSchedulingIgnoredDuringExecution":[{"weight":80,"preference":{"matchExpressions":[{"key":"disk","operator":"In","values":["ssd"]}]}]}}}}}}'
$ kubectl scale deploy soft --replicas=3 # label still gone...
$ kubectl get pods # 3/3 Running — preferred only VOTES, never vetoes
```

Proved: required = filter (Pending on mismatch), preferred = score (never blocks), **IgnoredDuringExecution** = running Pods stay. **Teardown:** `kubectl delete deploy soft`.

[↑ back to top](#)

**STEP 1 • Required anti-affinity on a ONE-node cluster**

```
$ kubectl create deploy sep --image=nginx --replicas=3
$ kubectl patch deploy sep --type=merge -p '{"spec":{"template":{"spec":{"affinity":{"podAntiAffinity":{"requiredDuringSchedulingIgnoredDuringExecution":[{"topologyKey":"kubernetes.io/hostname", "labelSelector":{"matchLabels":{"app":"sep"}}}]}}}}}}'
$ kubectl get pods
sep-aaa    1/1    Running    # fits — node is empty of "sep"
sep-bbb    0/1    Pending    # node already has one!
sep-ccc    0/1    Pending    # same hostname domain — no second one exists
```

WHY (events)

```
$ kubectl describe pod sep-bbb | tail -2
0/1 nodes are available:
1 node(s) didn't match pod
anti-affinity rules.
```

THE LESSON

```
# required anti-affinity:
# replicas > topology
# domains = permanent
# Pending. 3 replicas
# × 1 node = 2 orphans.
```

STEP 2 • Soften it — everyone fits

```
$ kubectl patch deploy sep --type=merge -p '{"spec":{"template":{"spec":{"affinity":{"podAntiAffinity":{"preferredDuringSchedulingIgnoredDuringExecution":[{"weight":100,"podAffinityTerm":{"topologyKey":"kubernetes.io/hostname", "labelSelector":{"matchLabels":{"app":"sep"}}}]}}}}}}'
$ kubectl get pods # 3/3 Running — co-located, but only because we must
```

Proved: topology domains are finite; `required` anti-affinity hard-caps your replica count per domain. **Teardown:** `kubectl delete deploy sep`.

[↑ back to top](#)



Setup note: preemption needs **resource pressure**. We fill the node with low-priority Pods holding real memory requests, then ask for a big high-priority Pod. Adjust sizes to your cluster's free memory.

STEP 1 · Two classes + a filled floor

```
$ kubectl create priorityclass low --value=100
$ kubectl create priorityclass high --value=1000
$ kubectl create deploy filler --image=polinux/stress --replicas=6 \
  -- stress --vm 1 --vm-bytes 10M --timeout 6000s
$ kubectl set resources deploy/filler --requests=memory=500Mi
$ kubectl patch deploy filler --type=merge -p \
  '{"spec":{"template":{"spec":{"priorityClassName":"low"}}}}'
$ kubectl get pods -l app=filler # 6 Running — floor is loaded
```

STEP 2 · The VIP arrives — preemption fires

```
$ kubectl run vip --image=polinux/stress --restart=Never \
  --overrides='{"spec":{"priorityClassName":"high","containers":[{"name":"s",
  "image":"polinux/stress","args":["--vm","1","--vm-bytes","50M"],
  "resources":{"requests":{"memory":"1500Mi"}}}]}}' -- stress --vm 1 --vm-bytes 50M
$ kubectl get pods -w
filler-xxx Terminating # ← a low-prio VICTIM was chosen
vip Running # ← took its place
```

STEP 3 · Read the verdict

```
$ kubectl get events | grep -i preempt
Preempted by pod default/vip ... to make room
$ kubectl get events | grep vip | head -3
Successfully assigned ... node minikube
```

Proved: scarcity + priority = eviction of the weak, *minimum victims only*. The filler Deployment recreates its Pod (Book 01 · CH 06) — everyone survives, the VIP just ate first. **Teardown:** `kubectl delete pod vip && kubectl delete deploy filler && kubectl delete pc low high`.

[↑ back to top](#)



STEP 1 · Cordon — close the door

```
$ kubectl cordon minikube
$ kubectl get nodes
NAME          STATUS          ROLES
minikube      Ready,SchedulingDisabled  control-plane,master
$ kubectl describe node minikube | grep Taints
node.kubernetes.io/unschedulable:NoSchedule # ← the cordon taint (CH 05!)
$ kubectl scale deploy web --replicas=3 # (any deployment)
$ kubectl get pods -w # 3rd pod → Pending: nowhere to land
```

STEP 2 · Drain — empty the room

```
$ kubectl drain minikube --ignore-daemonsets --delete-emptydir-data
node/minikube cordoned
evicting pod kube-system/coredns-...
evicting pod default/web-aaa
pod/web-aaa evicted
$ kubectl get pods -A | grep -v Running
# (single-node cluster: evicted pods go Pending — they have
# nowhere else to go. on a 3-node cluster they'd REAPPEAR
# on the healthy nodes — Book 01 · CH 06 in action.)
```

STEP 3 · Uncordon — reopen

```
$ kubectl uncordon minikube
$ kubectl get nodes # Ready — SchedulingDisabled gone
$ kubectl get pods -w # Pending pods schedule immediately 🎉
```

Proved: cordon = a NoSchedule taint you typed; drain = graceful, PDB-respecting evacuation; uncordon = instant refill.

Teardown: scale your deployment back and delete leftovers.



Muscle memory: `drain node --ignore-daemonsets --delete-emptydir-data` — the two flags ride together on virtually every real maintenance run.

↑ back to top



Q1. The Scheduler processes a Pod in which order?

- A. Score → Filter → Bind B. Filter → Score → Bind C. Bind → Filter → Score D. Filter → Bind → Score

Q2. Which taint effect evicts already-running Pods that don't tolerate it?

- A. NoSchedule B. PreferNoSchedule C. NoExecute D. None — taints never evict

Q3. `tolerationSeconds` applies to which effect?

- A. NoExecute B. NoSchedule C. PreferNoSchedule D. All three equally

Q4. A Pod with **required** node affinity for `disk=ssd` is running. The node's label is removed. The Pod...

- A. is evicted immediately B. reschedules to another node C. crashes D. keeps running — IgnoredDuringExecution

Q5. **required** pod anti-affinity on `kubernetes.io/hostname`, 3 replicas, a 1-node cluster:

- A. all 3 co-located B. 1 Running, 2 Pending C. the Deployment is rejected D. round-robin placement

Q6. `maxSkew` in a `topologySpreadConstraint` defines...

- A. minimum Pods per domain B. number of allowed domains C. max difference between the most- and least-loaded domains
D. the eviction threshold

Q7. A high-priority Pod is Pending for lack of memory. The Scheduler...

- A. may evict lower-priority Pods (minimum victims) to make room B. waits until a node frees naturally C. asks the kubelet to kill anything D. reboots the busiest node

Q8. `kubectl drain` refuses to run without `--ignore-daemonsets` because...

- A. DaemonSets restart too fast B. drain cannot evict privileged Pods C. kube-proxy blocks eviction D. DaemonSet Pods are node-bound by design and would just be recreated — drain skips them

Q9. Static Pods are...

- A. Pods that never restart B. created directly by the kubelet from a manifest directory — deleting via `kubectl` just gets them recreated C. Pods without IP addresses D. Deployments with `replicas=1`

Q10. After `kubectl cordon node1`, the Pods already running on node1...

- A. are evicted B. are terminated C. keep running; only NEW Pods avoid node1 D. migrate automatically

↑ back to top



Q	Ans	Why — one line
1	B	Filter (feasible) → Score (rank) → Bind — CH 01
2	C	NoExecute evicts existing non-tolerating Pods now — CH 03
3	A	The grace timer only makes sense for eviction — NoExecute — CH 04
4	D	Affinity is scheduling-time only; label changes never evict — CH 06
5	B	1 hostname domain < 3 replicas → extras Pending forever — CH 07, Lab 2
6	C	skew = max(domain count) – min(domain count), capped at maxSkew — CH 08
7	A	Preemption = scheduler-driven, minimum-victim eviction — CH 09, Lab 3
8	D	DaemonSet pods belong to their node; drain skips, flag acknowledges — CH 10
9	B	kubelet + manifest dir = source of truth; API deletion is overridden — CH 11
10	C	Cordon adds unschedulable taint — blocks placement, never evicts — CH 10, Lab 4



9–10

Ready for Book 05



6–8

Re-read missed chapters'  boxes

<6

Re-run Labs 1–4, then retake



Missed Q4 or Q10? They're the same fact wearing two hats: scheduling-time constraints never touch running Pods — only NoExecute taints and controller reconciliation do. Missed Q5? The topology-domain count is a hard ceiling for `required` anti-affinity.

[↑ back to top](#)



Filter / Score / Bind — the Scheduler's funnel: feasible → ranked → written to the Pod's nodeName.

Taint — node-side repulsion, `key=value:effect`; effects NoSchedule / PreferNoSchedule / NoExecute.

Toleration — Pod-side permission past a taint; operators Equal (exact) and Exists (wildcard, empty value).

tolerationSeconds — grace period before a NoExecute eviction takes effect.

Well-known taints — not-ready, unreachable, disk-pressure, memory-pressure, network-unavailable, unschedulable, control-plane — the cluster's automatic immune taints.

nodeSelector — simple hard label match on nodes.

Node affinity — required (filter) / preferred (score, weight 1–100) node matching with In/NotIn/Exists/DoesNotExist/Gt/Lt.

IgnoredDuringExecution — affinity evaluated at scheduling time only; later label changes never evict.

Pod (anti-)affinity — co-location / separation rules relative to other Pods, scoped by topologyKey.

topologyKey — the domain label defining "together": kubernetes.io/hostname, topology.kubernetes.io/zone|region.

topologySpreadConstraints — keep distribution even: maxSkew + topologyKey + whenUnsatisfiable (DoNotSchedule / ScheduleAnyway); empty domains attract.

PriorityClass — named integer priority; system-node-critical (2000001000) > system-cluster-critical (2000000000) > default 0.

Preemption — Scheduler evicts minimum lower-priority victims to place a higher-priority Pending Pod.

Cordon — mark node unschedulable (adds unschedulable:NoSchedule taint); no evictions.

Drain — cordon + graceful, PDB-respecting eviction of all recreatable Pods; `--ignore-daemonsets` / `--delete-emptydir-data` / `--force`.

PodDisruptionBudget (PDB) — "keep ≥ N replicas during voluntary disruptions"; drain waits for it.

Static Pod — kubelet-managed Pod from a node-local manifest directory (/etc/kubernetes/manifests); runs the control plane itself; undeletable via API.

Mirror Pod — the API-side read-only reflection kubelet creates so static Pods appear in kubectl.

[↑ back to top](#)



TAINTS & TOLERATIONS

```
$ kubectl taint node N gpu=nvidia:NoSchedule
$ kubectl taint node N gpu=nvidia:NoExecute
$ kubectl taint node N gpu=nvidia:NoSchedule- # remove
tolerations:
- {key: gpu, operator: Equal, value: nvidia, effect: NoSchedule}
- {key: gpu, operator: Exists, effect: NoExecute,
  tolerationSeconds: 300}
effects: block-new / prefer-avoid / evict-now
```

AFFINITY & SPREAD

```
node: required.../preferred...DuringScheduling
IgnoredDuringExecution
ops: In NotIn Exists DoesNotExist Gt Lt
pod: podAffinity / podAntiAffinity + topology
(hostname | zone | region)
topologySpreadConstraints:
- {maxSkew: 1, topologyKey: zone,
  whenUnsatisfiable: DoNotSchedule,
  labelSelector: {matchLabels: {app: web}}}
```

PRIORITY

```
$ kubectl create priorityclass hi --value=1000
spec: {priorityClassName: hi}
system-node-critical 2000001000
system-cluster-critical 2000000000
default pods: 0 · preemption = min victims
also drives kubelet eviction order (with QoS)
```

MAINTENANCE

```
$ kubectl cordon N
$ kubectl drain N --ignore-daemonsets \
  --delete-emptydir-data [--force] [--timeout=...]
# evicts gracefully, respects PDBs
# DaemonSet pods skipped
$ kubectl uncordon N
static pods: /etc/kubernetes/manifests
→ kubelet-born, API-undeletable
```

FACTS WORTH A JOB OFFER

Scheduling = Filter → Score → Bind; it happens ONCE — no re-evaluation of running Pods

Taints repel (node-side), affinity attracts (pod-side) — they never do each other's job

NoExecute = evict existing; tolerationSeconds = grace timer; Equal needs a value, Exists doesn't

required anti-affinity caps replicas at topology-domain count; spread constraints scale better

Preemption is minimum-victim arithmetic, not revenge; priority also orders kubelet evictions

Drain = cordon + PDB-respecting eviction; DaemonSet pods are skipped, not evicted

Static Pods bootstrap the control plane — the file on disk is the source of truth

[↑ back to top](#)



Book 01 built the machine, 02 dressed it, 03 connected it — Book 04 taught you **where everything is allowed to live, and how to move it safely.**

SCHEDULING – fully mapped:

└─ Funnel	filter (can) → score (should) → bind (done)
└─ Taints	NoSchedule · PreferNoSchedule · NoExecute
└─ Tolerations	Equal/Exists · tolerationSeconds
└─ Affinity	node + pod, required vs preferred, topologyKey
└─ Spread	maxSkew across zones – empty domains attract
└─ Priority	pecking order + preemption under scarcity
└─ Node ops	cordon → drain (PDB-aware) → uncordon · static pods

✓ You can now...

Explain any Pending Pod: read the event, name the failing stage, name the knob that fixes it

Dedicate nodes with taints + tolerations and pin workloads with affinity — without confusing the two directions

Design replica layouts for HA (spread/anti-affinity) and know exactly when `required` becomes a Pending trap

Set a priority hierarchy and predict who preempts whom under memory scarcity

Run the full maintenance dance on a production node and describe what each flag protects

Explain how the control plane boots itself with static Pods — and why you can't `kubectl-delete` them



Where to next — Book 05 · Storage: your Pods are placed perfectly — but `emptyDir` dies with the Pod (drain just taught you that the hard way). Book 05 gives data a life of its own: Volumes, PV/PVC, StorageClasses & dynamic provisioning, access modes, reclaim policies, StatefulSets with stable storage, and CSI. Your databases finally get a home that survives everything.

— End of Book 04 ·  —